



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
ESCOM

Trabajo Terminal

**Plataforma Web para la Denuncia Segura y Prevención de Riesgos
Digitales**

2026-B009

Presentan

Edgar Jair Flores Espinosa

Jose Bryan Omar Jimenez Velazquez

Leilani Shareni Santos Rivera

Directores

M. en C. Ulises Velez Saldaña

Dr. Jessie Paulina Guzmán Flores

Advertencia

“Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos con derecho de propiedad y por lo tanto, su uso quedará restringido a las aplicaciones que explícitamente se convengan.”

La aplicación no convenida exime a la escuela su responsabilidad técnica y da lugar a las consecuencias legales que para tal efecto se determinen.

Información adicional sobre este reporte técnico podrá obtenerse en:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional, situada en Av. Juan de Dios Bátiz s/n Teléfono: 57296000, extensión 52000.

Agradecimientos

A mis padres, por su apoyo incondicional y por ser el pilar fundamental en mi formación académica y personal.

A mis directores, el M. en C. Ulises Vélez Saldaña y la Dra. Jessie Paulina Guzmán Flores, por su invaluable guía, conocimiento y por confiar en este trabajo terminal.

Al Instituto Politécnico Nacional y a la Escuela Superior de Cómputo, por brindarme las herramientas y el entorno necesario para formarme como profesional.

*PONER LO QUE USTEDES QUIERAN, LO DE ARRIBA LO COPIE
JAJAJAJA*

Índice General

Glosario	VII
1. Marco Teórico	1
1.1. Problema de clasificación en el contexto de la DDP	1
1.1.1. Taxonomía de denuncias	2
1.1.2. Clasificación binaria y lógica de admisión	3
1.1.3. Ciclo de vida de los datos en el aprendizaje supervisado	4
1.2. Fundamentos de Procesamiento de Lenguaje Natural	5
1.2.1. Álgebra lineal en espacios vectoriales de alta dimensión	5
1.2.2. Preprocesamiento de la queja	6
1.3. Representación Vectorial del Texto	7
1.3.1. Modelos de Bolsa de Palabras y TF-IDF	7
1.3.2. Word Embeddings Estáticos — spaCy (es_core_news_lg)	8
1.3.3. Sentence Transformers — SBERT (paraphrase-multilingual-mpnet-base-v2)	9
1.4. Ingeniería de Características Híbridas	10
1.4.1. Codificación de variables normativas	11
1.4.2. Concatenación y Factor de Amplificación (α)	12
1.4.3. Simulación de producción: entrenamiento con metadatos e inferencia en modo autónomo	13
1.5. Modelos de Aprendizaje Automático Evaluados	14
1.5.1. Máquinas de Soporte Vectorial (SVM)	14
1.5.2. Regresión Logística	15
1.5.3. Ensamble de Árboles — Random Forest	16
1.5.4. Ensamble por Votación Suave	16
1.6. Estrategias de Entrenamiento y Validación Robusta	17
1.6.1. Validación Cruzada K -Fold Estratificada	17
1.6.2. Evaluación mediante Métricas de Inteligencia	18

1.6.3.	Carga Perezosa y Serialización con <code>joblib</code>	20
1.7.	Arquitecturas híbridas en sistemas distribuidos	21
1.7.1.	Arquitectura de Microservicios	21
1.7.2.	Arquitectura de tres capas	23
1.7.3.	Servidor web (NGINX)	24
1.8.	Tecnologías de Desarrollo	25
1.8.1.	Lenguaje de programación: Java	25
1.8.2.	Framework: Spring Boot	25
1.8.3.	Interfaces de Programación de Aplicaciones (API REST)	29
1.8.4.	Protocolos de Transporte Seguro: TLS 1.3	29
1.8.5.	Resiliencia del Sistema: API Gateway	30
1.9.	Infraestructura y Orquestación	31
1.9.1.	Virtualización e Hipervisores	31
1.9.2.	Máquinas Virtuales y Sistema Operativo Anfitrión	32
1.9.3.	Contenerización	33
1.9.4.	Docker	33
1.9.5.	Orquestación de Contenedores con Kubernetes	34
1.10.	Gestión de Bases de Datos	35
1.10.1.	PostgreSQL	36
1.10.2.	Manejo de datos semi-estructurados mediante JSONB	37
1.10.3.	Índices GIN y búsqueda eficiente de información	37
1.10.4.	Seguridad y cifrado de datos en reposo	38
1.10.5.	Escalabilidad y optimización de bases de datos	39

Índice de figuras

1.1. Comunicación entre microservicios.	22
1.2. Patrón de arquitectura de tres capas.	23
1.3. Endpoints de monitoreo proporcionados por Spring Boot Actuator.	27
1.4. Funcionamiento de Docker y Kubernetes en la infraestructura de microservicios. .	31

Índice de tablas

1.1. Features jurídicos estructurados derivados del Artículo 4°.	11
--	----

Glosario

Acuerdo de Conclusión: Documento final que cierra el expediente, pudiendo resultar en una recomendación, un cese de actos o un pronunciamiento de improcedencia.

Acuse de Recibo: Comprobante digital generado por el sistema que confirma que la DDP ha recibido la solicitud, otorgando un folio único de seguimiento.

Análisis de Patrones: Uso de algoritmos para identificar comportamientos repetitivos o anomalías (como focos de corrupción) dentro de grandes volúmenes de datos.

Análisis Bidimensional: Evaluación de datos cruzando dos variables distintas (ej. tipo de queja vs. ubicación geográfica) para obtener información estratégica.

Anonimato Bidireccional: Capacidad de un sistema para permitir la comunicación entre denunciante e investigador sin que ninguna de las partes conozca la identidad real de la otra.

API REST: Interfaz de programación de aplicaciones que utiliza peticiones HTTP para transferir datos entre componentes de software.

Arquitectura de Cero Conocimiento (Zero-Knowledge): Diseño de seguridad donde el servidor no tiene acceso a las llaves de cifrado del usuario, siendo incapaz de ver los datos almacenados.

Arquitectura de Microservicios: Estilo arquitectónico que estructura una aplicación como un conjunto de servicios pequeños, autónomos y acoplados de forma mínima.

Arquitectura Monolítica: Modelo de software donde todos los componentes del sistema están entrelazados en un solo programa, dificultando su escalabilidad.

AUC-ROC: Curva que mide la capacidad del modelo para distinguir entre clases; un área cercana a 1 indica un desempeño óptimo en la clasificación.

Autenticación Robusta: Proceso de validación de identidad que utiliza credenciales institucionales verificables (Boleta o Número de Empleado).

Autonomía Técnica: Capacidad de la DDP para emitir criterios y recomendaciones de forma independiente, sin influencia de otras autoridades.

Big Data: Conjuntos de datos tan grandes y complejos que requieren aplicaciones informáticas no tradicionales para su procesamiento.

Bolsa de Palabras (BoW): Técnica de representación de texto basada en la frecuencia de aparición de cada palabra, sin considerar el orden gramatical.

Caducidad / Rezago Administrativo: Situación que ocurre cuando un trámite no se realiza dentro de los plazos legales, afectando la eficiencia institucional.

Catálogo de Derechos Vulnerados: Listado estructurado de los derechos establecidos en la Ley Orgánica que sirve de base para la clasificación mediante PLN.

Cifrado de Extremo a Extremo (E2EE): Sistema de comunicación donde solo los usuarios finales pueden leer los mensajes, protegiendo la información de intermediarios.

Clasificador Neuro-simbólico: Enfoque de IA que combina redes neuronales (para aprendizaje) con lógica basada en reglas (para razonamiento).

Comunidad Politécnica: Conjunto de estudiantes, docentes, personal administrativo y egresados del IPN.

Competencia: Facultad o atribución legal que tiene un órgano (DDP) para conocer y resolver un asunto específico basado en su normativa.

Crowdsourcing: Modelo de obtención de servicios o ideas mediante la colaboración abierta de una comunidad o grupo de personas.

DDP (Defensoría de los Derechos Politécnicos): Órgano encargado de velar por los derechos de la comunidad del IPN.

Debido Proceso: Conjunto de formalidades esenciales que deben observarse en cualquier procedimiento legal para asegurar un resultado justo.

Disponibilidad: Capacidad de un sistema de permanecer operativo y accesible durante un periodo de tiempo determinado.

Escalabilidad: Propiedad de un sistema para manejar una cantidad creciente de trabajo o su potencial para ser ampliado.

Estado del Arte: Análisis de los conocimientos, tecnologías y soluciones más avanzados que existen actualmente sobre un tema de investigación.

Expediente Digital: Repositorio electrónico que centraliza documentación, evidencias multimedia y actuaciones jurídicas de un caso.

F1-Score: Métrica reina que representa el promedio balanceado entre Precisión y Recall, útil para evaluar la calidad global del modelo.

Flujo Operativo: Secuencia estandarizada de etapas por las que debe pasar una queja, desde su recepción hasta su archivo definitivo.

Georreferenciación / Geolocalización: Técnica que permite asignar una ubicación geográfica exacta (coordenadas) a un reporte o evento.

Grid Search: Búsqueda exhaustiva para encontrar los mejores hiperparámetros de un modelo probando todas las combinaciones posibles.

GRC (Governance, Risk, and Compliance): Estrategia para gestionar el gobierno corporativo, los riesgos y el cumplimiento legal de una organización.

Hiperplano: Frontera o muro matemático en un espacio multidimensional que separa las distintas clases (ej. quejas competentes de improcedentes).

Independencia Condicional: Supuesto del algoritmo Naive Bayes donde se asume que la presencia de una palabra no afecta a otra, simplificando el cálculo de probabilidades.

Infraestructura On-Premise: Modelo de despliegue donde los servidores y el software se encuentran físicamente dentro de las instalaciones de la institución.

Inferencia Bayesiana: Modelo estadístico de probabilidad que actualiza la estimación de un fenómeno a medida que se adquiere más evidencia.

Inmutabilidad de los Datos: Propiedad que asegura que la información registrada no puede ser alterada o borrada, garantizando la integridad del expediente.

Integridad Documental: Garantía de que los documentos que integran el expediente digital no han sido manipulados desde su carga original.

IPN (Instituto Politécnico Nacional): Institución pública mexicana de educación media superior, superior y posgrado.

Java Spring Boot: Framework diseñado para simplificar el desarrollo de aplicaciones Java basadas en microservicios.

Justicia Restaurativa: Enfoque de justicia centrado en reparar el daño causado a las víctimas, en lugar de solo castigar al infractor.

Ley Orgánica: Marco legal máximo que rige la estructura, fines y organización del Instituto Politécnico Nacional.

Manual de Organización / Procedimientos: Documentos normativos que definen las funciones y los pasos operativos internos de la Defensoría.

Margen: Distancia entre los vectores de soporte y el hiperplano; el sistema busca maximizar esta distancia para reducir errores de clasificación.

Matriz de Confusión: Tabla que permite visualizar el desempeño de un algoritmo, mostrando los aciertos y las confusiones (falsos positivos y negativos).

Máxima Verosimilitud (MLE): Método para estimar las probabilidades de un modelo basándose en la frecuencia de aparición de los datos en el entrenamiento.

Medidas de Protección: Acciones urgentes dictadas para salvaguardar la integridad de la persona quejosa durante la investigación.

Naive Bayes: Algoritmo probabilístico que clasifica quejas basándose en el contenido de sus palabras bajo un supuesto de independencia.

Oficio de Solicitud de Informe: Documento oficial para requerir a una autoridad que rinda su declaración sobre los hechos de una queja.

PLN / NLP (Procesamiento de Lenguaje Natural): Rama de la IA que permite a las máquinas entender, interpretar y manipular el lenguaje humano.

PostgreSQL: Sistema de gestión de bases de datos relacional orientado a objetos y de código abierto.

Precisión: Métrica que indica qué tan exacto es el sistema cuando clasifica una queja como positiva, evitando falsas alarmas.

Primer Contacto: Etapa inicial donde se recibe la solicitud y se realiza el filtrado preliminar de la queja.

Procedencia: Calificación de una solicitud que cumple con los requisitos legales para ser admitida y analizada de fondo.

Punto de Falla Único: Componente de un sistema que, en caso de fallar, provoca que todo el sistema deje de funcionar.

Quejoso: Persona que interpone una queja o denuncia por una presunta vulneración de sus derechos.

Radicación: Acto formal de registrar y asignar un número de expediente a una queja para iniciar su proceso legal.

RBAC (Role-Based Access Control): Método de control de acceso que restringe o permite funciones del sistema según el rol asignado al usuario.

Recall (Sensibilidad): Capacidad del sistema para detectar todos los casos positivos reales, asegurando que ninguna queja válida sea ignorada.

Reglas de Negocio: Políticas institucionales que el software debe ejecutar automáticamente para garantizar la validez del proceso.

SaaS (Software as a Service): Modelo de distribución de software donde los datos se alojan en servidores externos y se accede vía web.

Similitud de Coseno: Medida métrica utilizada para determinar qué tan parecidos son dos documentos de texto en un espacio vectorial.

SMOTE (Synthetic Minority Over-sampling Technique): Técnica de balanceo de datos que crea ejemplos sintéticos para clases minoritarias, permitiendo un aprendizaje parejo.

Soberanía de Datos: Principio que establece que los datos están sujetos a las leyes y control de la institución donde se generan.

Stemming y Lematización: Procesos lingüísticos para reducir palabras a su raíz o forma base (lema) para facilitar el análisis.

Suavizado de Laplace (α): Ajuste matemático que suma un valor pequeño a los conteos para evitar probabilidades de cero en palabras no vistas previamente.

SVM (Support Vector Machine): Algoritmo de aprendizaje supervisado que busca el hiperplano óptimo para clasificar datos en diferentes categorías.

Términos Procesales / Plazos Legales: Lapsos de tiempo obligatorios otorgados por la normativa para realizar una acción jurídica.

- TF-IDF:** Técnica de pesaje que resalta palabras clave de un problema y resta importancia a palabras comunes o irrelevantes.
- TLS (Transport Layer Security):** Protocolo criptográfico que proporciona comunicaciones seguras por red, cifrando los datos en tránsito.
- Tokenización:** Proceso de fragmentar una cadena de texto en unidades más pequeñas llamadas tokens (palabras o símbolos).
- Trazabilidad:** Capacidad de seguir el historial, la aplicación y la ubicación de un expediente a través de todas sus etapas.
- Triaje Administrativo:** Proceso de selección y priorización de quejas basado en urgencia, competencia legal y gravedad.
- Truco del Kernel (Kernel Trick):** Técnica que proyecta datos a dimensiones superiores para encontrar una separación lineal donde antes no era posible.
- Validación Cruzada (K-Fold):** Método de evaluación que divide los datos en K partes para asegurar que el sistema sea robusto con diferentes subconjuntos de datos.
- Variables de Holgura (ξ):** Parámetro de flexibilidad en un modelo SVM que permite que algunos puntos crucen la frontera para manejar datos con ruido.
- Vectores de Soporte:** Puntos o quejas que se encuentran más cerca de la frontera de decisión y definen la posición del hiperplano.
- Ventana de Contexto:** Rango de palabras que el modelo analiza alrededor de un término objetivo para aprender su significado semántico.
- Whistleblowing:** Acto de denunciar internamente o ante las autoridades actividades ilegales o poco éticas dentro de una organización.
- Word Embeddings:** Representación de palabras como vectores en un espacio multidimensional donde la cercanía geométrica indica cercanía semántica.
- Word2Vec / GloVe:** Modelos específicos utilizados para generar embeddings basados en el contexto local o estadísticas globales del texto.

Capítulo 1

Marco Teórico

Con el fin de optimizar el flujo operativo dentro de la Defensoría de Derechos Politécnicos (DDP), se implementará un sistema de clasificación para determinar la competencia o incompetencia de una queja. Este sistema brindará apoyo directamente en la etapa de primer contacto; con ello se busca transformar el análisis normativo en un modelo computacional capaz de identificar la competencia de manera consistente y eficiente.

1.1 Problema de clasificación en el contexto de la DDP

Este problema parte del marco legal presentado en el Acuerdo publicado en la Gaceta Politécnica, Número Extraordinario 622, donde se presenta a la DDP como el órgano encargado de promover, proteger y defender los derechos de la comunidad frente a actos u omisiones de las autoridades del Instituto.

En el Artículo 3° del mismo Acuerdo se establece que la Defensoría es competente para conocer quejas relacionadas con presuntas violaciones a los derechos de la comunidad politécnica; sin embargo, el Artículo 4° delimita los márgenes en las materias en que la Defensoría **no** es competente:

1. Afectaciones de carácter colectivo.
2. Resoluciones disciplinarias.
3. Derechos de naturaleza laboral.

4. Evaluaciones académicas de profesores, comisiones dictaminadoras o consejos técnicos consultivos escolares, salvo que en dicha evaluación se violen notoriamente los derechos de los interesados.
5. Procedimientos y resoluciones instaurados por el Órgano Interno de Control en el Instituto.

El proceso de recepción de quejas descrito en el Artículo 13 establece que el personal de la Defensoría debe determinar en primera instancia si la queja es competente o incompetente. Esta decisión, aparentemente sencilla, presenta en la práctica alta ambigüedad derivada de la redacción de los hechos. Dicha dependencia del criterio manual genera tres conflictos críticos:

1. El consumo excesivo de tiempo administrativo.
2. La variabilidad de interpretaciones ante casos similares.
3. La saturación operativa debida al incremento constante de solicitudes.

Con esto, el problema se define como una tarea de *Procesamiento de Lenguaje Natural* (PLN) y *clasificación supervisada*, donde el sistema debe aprender a identificar los patrones lingüísticos y normativos que separan una queja competente de una incompetente.

1.1.1 Taxonomía de denuncias

Lo anterior pareciera una tarea sencilla; no obstante, transformar una narrativa jurídica en un conjunto de datos procesable por algoritmos de aprendizaje automático requiere establecer una taxonomía clara que estructure el conocimiento normativo.

En el contexto de la Defensoría, las denuncias no se clasifican únicamente por su contenido textual, sino por una estructura jerárquica de validación que el sistema debe reconocer. Esta taxonomía se apoya en dos reglas críticas:

1. **Pertenencia a la comunidad y jerarquía.** Define si las personas involucradas pertenecen a la comunidad politécnica. El sistema deberá identificar la existencia de un quejoso con su estatus institucional y la autoridad a quien se imputa la violación. Sin esta validación de identidad, la queja carece de base para ser procesada.
2. **Exclusiones del Artículo 4°.** Constituye el núcleo del clasificador. Las denuncias se categorizan según la presencia de los cinco criterios de exclusión mencionados anteriormente. Esta regla se traduce en el sistema como siete campos binarios —características estructuradas— que procesan el análisis normativo:

- Naturaleza laboral vs. derechos politécnicos.
- Evaluación académica pura vs. vulneración de derechos en evaluación.
- Afectación individual vs. colectiva.
- Resolución disciplinaria vs. acto de autoridad.
- Procedimientos del OIC vs. acto u omisión de autoridad institucional.

Esta estructuración permite que el problema de clasificación no dependa únicamente del texto libre, sino de reglas establecidas en el Artículo 4°. Al final del proceso taxonómico, el sistema asigna a cada expediente una etiqueta binaria definitiva: *Competente* o *No Competente* (canalización a otra instancia o desechamiento), lo que constituye la variable de salida de los modelos de clasificación supervisada.

1.1.2 Clasificación binaria y lógica de admisión

Este clasificador se implementa bajo un esquema de **clasificación binaria**, decisión fundamentada directamente en el Artículo 13 del Acuerdo número extraordinario 622. A diferencia de otros sistemas de categorización temática —que discriminan entre dominios como deportes, política o finanzas— el clasificador de la DDP produce un veredicto dicotómico: *Competente* o *No Competente*.

Esta simplificación a dos clases responde a la lógica de admisión procesal por tres razones fundamentales:

1. **Naturaleza del flujo administrativo.** Dentro de la DDP, el primer contacto actúa como una compuerta. No existe un estado intermedio: una queja o es admitida para su investigación, o es desechada y canalizada a otra instancia. La clasificación binaria representa fielmente este proceso de toma de decisiones institucional.
2. **Optimización del entrenamiento.** Desde la perspectiva del aprendizaje automático, concentrar la muestra en dos etiquetas permite una mayor densidad de datos por categoría. Dado que el volumen de expedientes históricos etiquetados es finito, un modelo binario es más robusto y menos propenso al error que un modelo multiclase que intentara identificar simultáneamente el tipo específico de exclusión: laboral, académica o disciplinaria.
3. **Reducción del falso negativo.** La lógica de admisión prioriza la protección de los derechos de la comunidad. Al configurar el problema como binario, es posible ajustar el umbral de decisión del modelo —especialmente en algoritmos como SVM— para minimizar los falsos

negativos, asegurando que ninguna queja potencialmente competente sea descartada por una clasificación errónea.

Por lo tanto, si el modelo predice la clase *No Competente*, el sistema asiste al personal sugiriendo la fundamentación legal basada en el Artículo 4°. Si la predicción es *Competente*, el expediente fluye automáticamente hacia la etapa de radicación en la Subdefensoría, logrando así la meta de reducir los tiempos de respuesta institucional.

1.1.3 Ciclo de vida de los datos en el aprendizaje supervisado

La implementación de un clasificador inteligente requiere establecer un flujo técnico robusto que garantice que la narrativa jurídica se transforme en una señal numérica apta para el aprendizaje automático. En el contexto de este proyecto, el ciclo de vida de los datos se estructura en seis etapas fundamentales:

1. **Captura de datos.** El ciclo inicia en la interfaz de registro donde el quejoso proporciona de forma estructurada su identidad institucional y, de forma no estructurada, el relato de los hechos. Esta etapa asegura que la entrada cumpla con los requisitos mínimos de admisión definidos en el Artículo 13.
2. **Preprocesamiento y limpieza.** La narrativa libre se somete a técnicas de PLN. Se eliminan elementos de ruido, se normaliza el texto y se identifican términos clave que corresponden a las exclusiones normativas del Artículo 4°.
3. **Vectorización y enriquecimiento.** El texto se convierte en representaciones vectoriales (*embeddings*). Simultáneamente, el sistema inyecta las características jurídicas binarias extraídas del expediente, aplicando el factor de amplificación α para dar peso a la señal normativa sobre la semántica.
4. **Entrenamiento y ajuste.** Utilizando un conjunto de datos histórico previamente etiquetado, el modelo aprende a identificar las fronteras de decisión legal. Se emplean técnicas de validación cruzada para medir la robustez de la generalización.
5. **Inferencia y sugerencia.** Ante una queja nueva, el modelo procesa los vectores y genera una predicción de competencia acompañada de un nivel de probabilidad. El sistema asiste al personal humano mostrando el veredicto sugerido y la posible fundamentación legal.
6. **Retroalimentación.** El ciclo se cierra cuando el personal jurídico valida o corrige la sugerencia del modelo. Estas decisiones se reincorporan al repositorio de datos para el reentrenamiento

futuro, permitiendo que el clasificador evolucione conforme cambian los criterios institucionales.

Este ciclo asegura que el sistema no opere como una herramienta estática, sino que mantenga la trazabilidad y la integridad de la información jurídica en cada una de sus fases.

1.2 Fundamentos de Procesamiento de Lenguaje Natural

El Procesamiento de Lenguaje Natural (PLN) es la rama de la inteligencia artificial que dota a las computadoras de la capacidad de comprender, interpretar y generar lenguaje humano. En el contexto jurídico, el PLN enfrenta desafíos particulares: el vocabulario especializado, la estructura argumentativa implícita y la alta dependencia del contexto normativo hacen que dos quejas léxicamente similares puedan tener clasificaciones opuestas.

1.2.1 Álgebra lineal en espacios vectoriales de alta dimensión

El fundamento matemático sobre el que descansan todas las representaciones textuales empleadas en este sistema es el **espacio vectorial de alta dimensión**. La idea central es que cualquier documento puede representarse como un punto en un espacio $\mathbb{R}^d$, donde d es el número de dimensiones del vector. Documentos semánticamente similares ocupan regiones cercanas en dicho espacio.

Sea $\mathbf{v} \in \mathbb{R}^d$ un vector de representación de un documento. La **similitud coseno** entre dos documentos $\mathbf{u}$ y $\mathbf{v}$ se define como:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \cdot \sqrt{\sum_{i=1}^d v_i^2}} \quad (1.1)$$

donde $\text{sim} \in [-1, 1]$. Un valor cercano a 1 indica alta similitud semántica; esta métrica es la base del razonamiento por proximidad que subyace a todos los vectorizadores empleados en el sistema.

En este proyecto se trabaja con espacios de $d = 300$ dimensiones (spaCy) y $d = 768$ dimensiones (Sentence-BERT). La diferencia en dimensionalidad no es trivial: a mayor dimensión, el modelo

puede capturar matices semánticos más finos, pero también requiere mayor capacidad computacional y datos suficientes para evitar la *maldición de la dimensionalidad* (el volumen del espacio aumenta exponencialmente con cada nueva dimensión, lo que provoca que los datos se vuelvan muy dispersos).

Las operaciones fundamentales que el sistema ejecuta sobre estos espacios son:

- **Suma de vectores:** permite combinar representaciones de tokens individuales en una representación de documento.
- **Promedio ponderado:** base del vectorizador de spaCy.
- **Producto punto y norma:** usados internamente por SVM para calcular márgenes de separación.
- **Concatenación horizontal:** operación central de la arquitectura híbrida para unir embeddings de texto con features jurídicos.

1.2.2 Preprocesamiento de la queja

Antes de convertir el texto en vectores, la narrativa libre del quejoso debe pasar por una cadena de transformaciones que reduzcan el ruido y estandaricen la representación lingüística. Este proceso es especialmente relevante en textos jurídicos, donde variaciones ortográficas, errores tipográficos o sinónimos coloquiales de términos normativos pueden afectar la calidad de los embeddings.

Tokenización

La **tokenización** segmenta el texto continuo en unidades mínimas de significado (tokens). spaCy realiza una tokenización sensible al contexto que distingue, por ejemplo, contracciones y signos de puntuación:

"me negó el acceso" → [me, negó, el, acceso]

Lematización

La **lematización** reduce cada token a su forma canónica (*lema*), eliminando variaciones morfológicas:

negaron → negar, discriminado → discriminar, estudiantiles → estudiantil

Filtrado de palabras vacías

Las **palabras vacías** (*stop words*) son términos de alta frecuencia y bajo contenido semántico (artículos, preposiciones, conjunciones). En textos jurídicos, sin embargo, ciertas palabras que en otros contextos serían vacías — como “no”, “sin” o “nunca” — pueden ser jurídicamente decisivas. spaCy permite personalizar esta lista para el dominio específico de la Defensoría.

Análisis morfosintáctico con spaCy

El modelo `es_core_news_lg` de spaCy realiza, además de la tokenización y lematización, un análisis morfosintáctico completo que asigna a cada token su categoría gramatical (*POS tagging*) y sus dependencias sintácticas. Esto permite identificar, por ejemplo, que en la oración:

“El director me negó el acceso al laboratorio”

el sujeto es *director* (autoridad), el verbo es *negar* (acto de omisión) y el complemento es *acceso al laboratorio* (derecho afectado). Esta información estructural enriquece la representación semántica más allá del léxico superficial.

1.3 Representación Vectorial del Texto

La representación vectorial del texto es el puente entre el lenguaje natural y los algoritmos de aprendizaje automático. A continuación se presenta la evolución de estas técnicas desde los modelos estadísticos hasta los modelos de lenguaje contextuales de última generación.

1.3.1 Modelos de Bolsa de Palabras y TF-IDF

La caracterización de bolsa de palabras puede ser considerado una forma de procesamiento de texto sencilla por su aparente simplicidad en recuento de palabras en un conjunto de textos. En bolsa de palabras, cada palabra individual se convierte en una dimensión separada del espacio vectorial. Si un conjunto de texto tiene n cantidad de palabras, el espacio vectorial resultante tendrá n dimensiones. El modelo trazará cada documento de texto por separado como un punto en el espacio vectorial. La posición de un punto está determinada por la cantidad de veces que aparece la palabra de esa dimensión dentro del documento del punto.

Los modelos de **Bolsa de Palabras** (*Bag of Words*, BoW) son el punto de partida histórico de la representación vectorial. Representan un documento como un vector de frecuencias de términos sobre el vocabulario total del corpus:

$$\mathbf{v}_{BoW}(d) = [f(t_1, d), f(t_2, d), \dots, f(t_V, d)] \quad (1.2)$$

donde $f(t_i, d)$ es la frecuencia del término t_i en el documento d y V es el tamaño del vocabulario.

TF-IDF (*Term Frequency – Inverse Document Frequency*) refina este enfoque ponderando cada término según su frecuencia en el documento y su rareza en el corpus:

$$\text{tf-idf}(t, d, D) = \underbrace{\frac{f(t, d)}{\sum_{t'} f(t', d)}}_{\text{TF}} \times \underbrace{\log \frac{|D|}{|\{d' \in D : t \in d'\}|}}_{\text{IDF}} \quad (1.3)$$

Limitaciones críticas para el dominio jurídico. Aunque son computacionalmente eficientes, estos modelos son inadecuados para clasificar quejas de la DDP por dos razones fundamentales:

1. **Pérdida del orden y contexto:** Las frases “*el director negó el acceso*” y “*el acceso negó al director*” generan el mismo vector, aunque tienen significados radicalmente distintos.
2. **Sesgo léxico superficial:** El término “*calificación*” tendría alta frecuencia tanto en quejas por evaluación académica (No Competente) como en quejas por extorsión académica (Competente), haciendo imposible la distinción correcta.

Estos modelos se presentan como antecedentes necesarios para contextualizar la superioridad de los enfoques basados en embeddings empleados en este sistema.

1.3.2 Word Embeddings Estáticos — spaCy (es_core_news_lg)

Los **embeddings estáticos** resuelven la limitación semántica de BoW al representar cada palabra como un vector denso en $\mathbb{R}^{300}$, entrenado sobre grandes corpus en español mediante el algoritmo *word2vec*.^[65] Palabras semánticamente relacionadas —como *acoso* y *hostigamiento*— ocupan regiones cercanas en este espacio.

El modelo `es_core_news_lg` de spaCy [66] fue entrenado sobre el corpus *Spanish Web Crawl* y genera vectores de 300 dimensiones por token. La representación de un documento completo se obtiene promediando los vectores de los tokens informativos:

$$\mathbf{v}_{spaCy}(d) = \frac{1}{|T|} \sum_{t \in T} \mathbf{w}_t, \quad T = \{t \in d \mid \text{has_vector}(t) \wedge \neg \text{is_stop}(t)\} \quad (1.4)$$

donde $\mathbf{w}_t \in \mathbb{R}^{300}$ es el vector del token t y T es el conjunto de tokens informativos (con vector disponible y no vacías).

Al promediar todos los vectores del documento se pierde el orden sintáctico y el contexto local. La frase “*el profesor me acosó condicionando la calificación*” y la frase “*el profesor condicionó la calificación — sin acosar*” generarían vectores cercanos, a pesar de tener clasificaciones jurídicas distintas.

1.3.3 Sentence Transformers — SBERT (paraphrase-multilingual-mpnet-base-v2)

Sentence-BERT (SBERT) representa el estado del arte en codificación semántica de textos.[64] A diferencia de los embeddings estáticos, SBERT genera un único vector de representación para toda la oración, capturando las relaciones entre palabras a lo largo del texto completo mediante la arquitectura *Transformer*.

Arquitectura

Transformer

El bloque central del Transformer es el mecanismo de **atención multi-cabeza** (*multi-head self-attention*), que calcula para cada token su relevancia en función de todos los demás tokens del documento:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1.5)$$

donde Q , K y V son las matrices de consulta, clave y valor derivadas de los embeddings de entrada, y d_k es la dimensión de la clave.[63] Este mecanismo permite que el modelo asigne mayor peso a los tokens contextualmente relevantes. Por ejemplo, en la queja:

“Me reprobó porque no acepté salir con él”

el mecanismo de atención aprende a conectar *reprobó* con *acepté salir*, identificando que la relación causal entre ambos constituye un acto de acoso, no una evaluación académica pura.

Entrenamiento contrastivo de SBERT

SBERT extiende BERT con una capa de *pooling* sobre los estados ocultos y se entrena mediante aprendizaje contrastivo con pares de oraciones semánticamente equivalentes y no equivalentes:

$$\mathcal{L}_{contrastiva} = \sum_{(i,j)} [\text{sim}(\mathbf{e}_i, \mathbf{e}_j) - y_{ij}]^2 \quad (1.6)$$

donde $y_{ij} = 1$ si las oraciones i y j son semánticamente equivalentes y $y_{ij} = 0$ en caso contrario.

Modelo empleado

El modelo *paraphrase-multilingual-mpnet-base-v2* fue entrenado sobre más de 50 idiomas, incluido el español. Genera embeddings de **768 dimensiones** por documento y ha demostrado superioridad frente a los promedios de *word embeddings* en tareas de clasificación de texto corto y mediano. Esta capacidad es crítica para distinguir los casos límite de la DDP, donde el acto jurídicamente relevante puede estar expresado en una sola oración dentro de un párrafo más extenso.

1.4 Ingeniería de Características Híbridas

Una de las contribuciones centrales de este sistema es la **arquitectura híbrida** que combina la representación semántica del texto con conocimiento normativo estructurado. Esto responde a una limitación fundamental de los modelos puramente textuales: pueden aprender correlaciones léxicas espurias en lugar de la lógica jurídica subyacente.

1.4.1 Codificación de variables normativas

Los cinco criterios de incompetencia del Artículo 4° se traducen en **siete variables binarias** (features jurídicos), que codifican el análisis normativo de cada queja de forma estructurada:

Tabla 1.1: Features jurídicos estructurados derivados del Artículo 4°.

# Variable	Descripción normativa
1 es_conflicto_laboral_art4_I	El fondo del conflicto es una relación laboral o sindical (salarios, plazas, escalafón, prestaciones).
2 es_evaluacion_academica_pura_art4_II	La queja versa <i>únicamente</i> sobre la valoración del aprendizaje por parte de un profesor o comisión.
3 hay_vulneracion_notoria_en_evaluacion	Existe una violación de derechos dentro de un proceso evaluativo (extorsión, acoso, discriminación).
4 es_resolucion_disciplinaria_art4_III	La queja impugna una resolución disciplinaria ya emitida.
5 es_afectacion_colectiva_art4_IV	La afectación descrita es grupal, no individual.
6 involucra_oic_art4_V	El asunto está siendo conocido por el Órgano Interno de Control.
7 hay_acto_u_omision_autoridad	Existe un acto u omisión de una autoridad institucional que vulnera derechos políticos.

Estas variables se representan como un vector $\mathbf{f} \in \{0, 1\}^7$ y se determinan a partir del análisis del expediente, ya sea de forma manual durante la construcción del corpus o de forma automática en una etapa futura mediante un modelo extractor.

La lógica de consistencia entre las variables y la clase asignada es la siguiente:

- Si clase = No Competente por razón laboral: es_conflicto_laboral_art4_I = 1, resto = 0.

- Si `clase = Competente`: `hay_acto_u_omision_autoridad = 1`, todas las fracciones del Art. 4° = 0.
- La variable 3 (`hay_vulneracion_notoria`) solo puede ser 1 cuando la variable 2 (`es_evaluacion_academica_pura`) también es 1 y la clase es *Competente*: esto modela la excepción de la fracción IV del Artículo 4°.

1.4.2 Concatenación y Factor de Amplificación (α)

La unión del embedding semántico $\mathbf{e} \in \mathbb{R}^d$ con el vector jurídico $\mathbf{f} \in \{0, 1\}^7$ se realiza mediante **concatenación horizontal**:

$$\mathbf{x}_{final} = [\mathbf{e} \parallel \alpha \cdot \mathbf{f}] \in \mathbb{R}^{d+7} \quad (1.7)$$

donde $d \in \{300, 768\}$ según el vectorizador empleado y α es el **factor de amplificación**.

Justificación matemática del factor α

Sin amplificación ($\alpha = 1$), los 7 features jurídicos quedan efectivamente opacados por las d dimensiones del embedding, cuya norma promedio es significativamente mayor. Para ilustrarlo, considérese la contribución relativa de cada componente a la norma del vector concatenado:

$$\frac{\|\alpha \cdot \mathbf{f}\|}{\|\mathbf{e}\| + \|\alpha \cdot \mathbf{f}\|} = \frac{\alpha\sqrt{7}}{\|\mathbf{e}\| + \alpha\sqrt{7}} \quad (1.8)$$

Con $\|\mathbf{e}\| \approx 15$ (valor típico para embeddings de Sentence-BERT normalizados) y $\alpha = 1$:

$$\frac{1 \cdot \sqrt{7}}{15 + \sqrt{7}} \approx \frac{2,65}{17,65} \approx 15 \%$$

Con $\alpha = 3$:

$$\frac{3\sqrt{7}}{15 + 3\sqrt{7}} \approx \frac{7,94}{22,94} \approx 35 \%$$

El valor $\alpha = 3,0$ fue seleccionado experimentalmente como el punto de equilibrio donde la señal jurídica tiene peso suficiente para influir en las fronteras de decisión del clasificador, sin dominar completamente sobre la información semántica del texto.

1.4.3 Simulación de producción: entrenamiento con metadatos e inferencia en modo autónomo

Una consideración de diseño fundamental del sistema es la asimetría entre el escenario de entrenamiento y el escenario de producción. Durante la construcción del corpus, el personal jurídico anota manualmente los siete atributos booleanos del Artículo 4° para cada expediente, lo que enriquece la señal de aprendizaje. Sin embargo, en el momento de la inferencia, el sistema recibe únicamente el relato libre del quejoso: no existe un experto que complete los atributos en tiempo real.

Esta asimetría se modela mediante dos modos de operación diferenciados:

1. **Modo asistido.** El personal de primer contacto completa el formulario con los atributos jurídicos mientras atiende al quejoso. El clasificador opera con el vector completo $\mathbf{x}_{final} = [\mathbf{e} \parallel \alpha \cdot \mathbf{f}]$, aprovechando tanto la semántica del texto como la señal normativa estructurada.
2. **Modo autónomo.** Los atributos jurídicos se fijan en cero ($\mathbf{f} = \mathbf{0}$), de forma que el clasificador basa su veredicto exclusivamente en la representación semántica del texto: $\mathbf{x}_{final} = [\mathbf{e} \parallel \mathbf{0}]$. Este modo representa el escenario de producción real y constituye la condición de estrés bajo la cual se evalúa la robustez de los modelos.

La distinción entre ambos modos tiene implicaciones directas para la selección del clasificador óptimo: un modelo que dependa excesivamente de la señal normativa amplificada puede exhibir métricas perfectas durante el entrenamiento, pero degradarse significativamente en producción cuando dicha señal no está disponible. Por esta razón, el criterio de selección del modelo final privilegia el desempeño en modo autónomo sobre el desempeño con metadatos.

1.5 Modelos de Aprendizaje Automático Evaluados

Para garantizar la selección del clasificador óptimo para el dominio jurídico de la DDP, se evaluaron cuatro configuraciones de algoritmos de aprendizaje supervisado con distintos paradigmas de construcción de fronteras de decisión.

1.5.1 Máquinas de Soporte Vectorial (SVM)

Las **Máquinas de Soporte Vectorial** (*Support Vector Machines*, SVM) son algoritmos de clasificación que buscan el **hiperplano de máximo margen** que separa las clases en el espacio de características [61].

Dado un conjunto de entrenamiento $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ con $y_i \in \{-1, +1\}$, la SVM resuelve el problema de optimización:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.a.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (1.9)$$

donde C es el parámetro de regularización que controla la tolerancia al error de clasificación ($C = 0,5$ en este sistema) y ξ son variables de holgura que permiten clasificación blanda.

Kernel

RBF

Para datos no linealmente separables en el espacio original, la SVM proyecta los datos a un espacio de mayor dimensión mediante la **función de kernel**. El kernel de Base Radial (*Radial Basis Function*, RBF) empleado en este sistema calcula la similitud entre dos puntos como:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right) \quad (1.10)$$

donde γ controla el radio de influencia de cada vector de soporte. El kernel RBF es especialmente adecuado para embeddings de alta dimensión porque puede construir fronteras de decisión no

lineales arbitrariamente complejas, adaptándose a la distribución de los casos jurídicos en el espacio semántico.

Ventaja en el dominio jurídico

La SVM con kernel RBF maximiza el margen de separación entre las clases, lo que le confiere una robustez natural ante casos límite: quejas que caen en la frontera entre *Competente* y *No Competente* se clasifican según el hiperplano de máximo margen, no según la densidad local de puntos. No obstante, esta ventaja geométrica está condicionada a la disponibilidad de una señal de entrada suficientemente discriminativa; cuando los atributos jurídicos no están presentes, la eficacia del clasificador lineal depende enteramente de la calidad del embedding semántico.

1.5.2 Regresión

Logística

La **Regresión Logística** es un modelo de clasificación lineal que estima la probabilidad de pertenencia a la clase positiva mediante la función logística (sigmoide):

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \quad (1.11)$$

Los parámetros $\mathbf{w}$ y b se estiman minimizando la función de pérdida de entropía cruzada binaria:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] + \lambda \|\mathbf{w}\|^2 \quad (1.12)$$

donde λ es el término de regularización L2. La configuración empleada en este sistema es `max_iter=1000` con $C = 0,3$.

Interpretabilidad. La Regresión Logística produce coeficientes w_j directamente interpretables: un coeficiente positivo asociado a una dimensión del embedding indica que ese componente favorece la clasificación como *Competente*. Esta propiedad es valiosa en el contexto jurídico, donde la explicabilidad de la decisión puede ser requerida institucionalmente.

1.5.3 Ensamble de Árboles — Random Forest

El **Random Forest** es un método de ensamble que construye múltiples árboles de decisión sobre subconjuntos aleatorios de los datos y las características, promediando sus predicciones para reducir la varianza [62]:

$$\hat{y}_{RF}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}) \quad (1.13)$$

donde B es el número de árboles ($B = 300$ en este sistema) y T_b es el b -ésimo árbol entrenado sobre una muestra bootstrap.

Partición del espacio de características. Los árboles de decisión particionan el espacio de características con cortes ortogonales a los ejes, construyendo regiones de decisión rectangulares de forma recursiva. Aunque en espacios de dimensión muy alta esta estrategia puede ser menos eficiente que el hiperplano de máximo margen de la SVM, el promedio de un número grande de árboles con submuestreo aleatorio de características reduce significativamente la varianza y confiere al Random Forest una notable capacidad de generalización incluso cuando la representación semántica del texto no está apoyada por atributos estructurados.

1.5.4 Ensemble por Votación Suave

La **votación suave** (*soft voting*) es una estrategia de *ensemble* que combina las probabilidades predichas por múltiples clasificadores base, promediándolas para obtener una predicción final más estable [?]:

$$\hat{y} = \arg \max_c \frac{1}{M} \sum_{m=1}^M p_m(c | \mathbf{x}) \quad (1.14)$$

donde M es el número de modelos base y $p_m(c | \mathbf{x})$ es la probabilidad que el modelo m asigna a la clase c . A diferencia de la **votación dura** —que cuenta votos de clase— la votación suave aprovecha la magnitud de la confianza de cada clasificador, lo que generalmente produce predicciones más calibradas cuando los clasificadores base tienen tasas de error no correlacionadas.

En este sistema, el *ensemble* integra SVM-RBF, Regresión Logística y Random Forest. La motivación es que cada clasificador tiene fortalezas complementarias: SVM maximiza el margen geométrico, la Regresión Logística produce probabilidades linealmente calibradas, y Random Forest captura fronteras no lineales por partición del espacio. Si sus errores no están correlacionados, el *ensemble* reduce la varianza de la predicción individual respecto a cualquiera de los modelos base.

Para que el `VotingClassifier` pueda operar en modo `voting="soft"`, todos los estimadores deben exponer el método `predict_proba`. Dado que SVC no lo implementa de forma nativa, se envuelve en `CalibratedClassifierCV`, que estima probabilidades mediante validación cruzada interna sobre el conjunto de entrenamiento.

1.6 Estrategias de Entrenamiento y Validación Robusta

La selección y evaluación de los modelos requiere estrategias que garanticen que las métricas reportadas reflejan la capacidad de generalización real del sistema y no son artefactos de una partición particular de los datos.

1.6.1 Validación Cruzada *K*-Fold Estratificada

La **validación cruzada *K*-fold** divide el conjunto de datos en K particiones (*folds*) de tamaño aproximadamente igual. En cada iteración, $K - 1$ particiones se usan para entrenamiento y la restante para evaluación, rotando hasta que todos los datos hayan sido usados como conjunto de prueba:

$$\overline{F1}_{CV} = \frac{1}{K} \sum_{k=1}^K F1_k \quad (1.15)$$

La variante **estratificada** garantiza que la proporción de clases se mantiene en cada pliegue, lo cual es crítico ante corpus con posible desbalance entre quejas competentes e incompetentes.

En este proyecto se empleó $K = 5$, lo que significa que en cada iteración el 80 % de los datos se usa para entrenamiento y el 20 % restante para evaluación. La desviación estándar entre pliegues (σ_{CV}) se usa como medida de **estabilidad** del modelo:

$$\sigma_{CV} = \sqrt{\frac{1}{K} \sum_{k=1}^K (F1_k - \overline{F1}_{CV})^2} \quad (1.16)$$

Un σ_{CV} cercano a cero indica que el modelo no es sensible a la partición específica de los datos, condición deseable para un sistema de apoyo en producción.

Nota de implementación. En la implementación de este proyecto, la validación cruzada $K = 5$ se aplica exclusivamente sobre el conjunto de entrenamiento (70 % del corpus, 367 casos), no sobre el dataset completo. Esto significa que cada *fold* evalúa aproximadamente 73 casos internos del entrenamiento. Esta validación se complementa con un conjunto *holdout* independiente del 15 % (79 casos), completamente aislado durante el entrenamiento y la selección de hiperparámetros, cuyo F1 ponderado —evaluado en modo autónomo, sin atributos jurídicos— constituye el criterio definitivo de selección del modelo.

1.6.2 Evaluación mediante Métricas de Inteligencia

Accuracy

Antes de definir las métricas, es necesario establecer los cuatro resultados posibles de una clasificación binaria, organizados en la **matriz de confusión**:

- **Verdadero Positivo (TP):** la queja es competente y el modelo la clasifica como competente.
- **Verdadero Negativo (TN):** la queja es incompetente y el modelo la clasifica como incompetente.
- **Falso Positivo (FP):** la queja es incompetente pero el modelo la clasifica como competente.
- **Falso Negativo (FN):** la queja es competente pero el modelo la clasifica como incompetente. En el contexto de la DDP, este es el error más grave: implica descartar una queja que debía ser atendida.

La **exactitud** (*accuracy*) mide la proporción de predicciones correctas sobre el total de instancias:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1.17)$$

En el contexto de la DDP, donde las consecuencias de un falso negativo (queja competente clasificada como incompetente) son más graves que las de un falso positivo, la accuracy por sí sola no es suficiente.

Precisión, **Recall** **y** **F1-Score**

$$\text{Precisión} = \frac{TP}{TP + FP} \quad (1.18)$$

La **Precisión** mide qué proporción de las quejas clasificadas como competentes realmente lo son; penaliza las falsas alarmas.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (1.19)$$

El **Recall** mide qué proporción de las quejas realmente competentes fueron identificadas correctamente; penaliza los casos omitidos. En el contexto de la DDP el Recall tiene mayor peso institucional: un Recall bajo implica que quejas legítimas están siendo descartadas sin revisión.

$$F1 = \frac{2 \cdot \text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}} \quad (1.20)$$

El **F1-Score** es la media armónica entre Precisión y Recall, ofreciendo un balance que penaliza los extremos donde una métrica es alta a costa de la otra.

$$F1 = \frac{2 \cdot \text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}} \quad (1.21)$$

El **F1 ponderado** (*weighted F1*) extiende este cálculo a ambas clases, ponderando por el número de instancias de cada una:

$$F1_w = \sum_{c \in C} \frac{n_c}{N} \cdot F1_c \quad (1.22)$$

donde n_c es el número de instancias de la clase c y N el total. Esta métrica es la principal para la selección del modelo óptimo en este sistema.

Análisis de la varianza σ

La desviación estándar del F1 en validación cruzada (σ_{CV}) complementa el valor medio al revelar la estabilidad del modelo. Un modelo con $\overline{F1}_{CV} = 0,98$ y $\sigma_{CV} = 0,04$ puede ser menos confiable que uno con $\overline{F1}_{CV} = 0,97$ y $\sigma_{CV} = 0,00$, pues el primero es sensible a la distribución particular de los datos en cada pliegue.

1.6.3 Carga Perezosa y Serialización con joblib

La **carga perezosa** (*lazy loading*) es un patrón de diseño donde los recursos computacionalmente costosos —como la carga del modelo de Sentence-BERT o el modelo de spaCy— se inician únicamente en el momento en que son requeridos por primera vez, no al inicio del programa. Esto reduce el tiempo de arranque del sistema y el consumo de memoria en instancias donde solo se requiere predicción.

La **serialización** de los pipelines se realiza mediante la biblioteca `joblib`, que convierte el objeto Python completo —vectorizador y clasificador juntos— en un archivo binario `.pkl`:

Pipeline(vectorizador, clasificador) $\xrightarrow{\text{joblib.dump}}$ `modelo.pkl` $\xrightarrow{\text{joblib.load}}$ Pipeline listo para inferencia

Esta estrategia es técnicamente relevante por tres razones:

1. **Integridad:** Vectorizador y clasificador se serializan juntos, eliminando el riesgo de usar un clasificador entrenado con un vectorizador distinto al de producción.
2. **Eficiencia:** `joblib` emplea compresión paralela y mapeo de memoria (*memory mapping*), resultando significativamente más rápido que `pickle` estándar para arrays numéricos de gran tamaño como los parámetros internos de Sentence-BERT.
3. **Reproducibilidad:** El archivo `.pkl` incluye la semilla aleatoria (`random_state=42`) y todos los hiperparámetros, garantizando que las predicciones son idénticas entre sesiones y entornos de ejecución.

La convención de nomenclatura de los archivos serializados es:

`<vectorizador>__<clasificador>__<timestamp>.pkl`

lo que permite trazabilidad temporal de los modelos entrenados y facilita la comparación y auditoría de versiones en el contexto institucional de la DDP.

1.7 Arquitecturas híbridas en sistemas distribuidos

Las arquitecturas híbridas representan un enfoque de diseño de software que combina distintos estilos arquitectónicos dentro de un mismo ecosistema computacional con el objetivo de aprovechar las fortalezas de cada modelo. En sistemas distribuidos modernos es común integrar arquitecturas de microservicios con patrones internos de separación por capas, permitiendo desacoplar tanto los servicios como la lógica interna de cada componente.

Este enfoque favorece la modularidad, la mantenibilidad y la escalabilidad del sistema, ya que cada servicio puede operar de forma independiente mientras mantiene una organización estructurada de sus responsabilidades internas. Asimismo, la combinación de múltiples patrones arquitectónicos permite construir sistemas capaces de adaptarse a entornos de alta concurrencia y procesamiento distribuido.

1.7.1 Arquitectura de Microservicios

La arquitectura de microservicios es un enfoque de desarrollo de software en el que una aplicación se descompone en un conjunto de servicios pequeños, autónomos y altamente cohesionados. Cada servicio representa una capacidad funcional específica y se ejecuta como un proceso independiente, comunicándose con otros componentes mediante mecanismos ligeros, generalmente APIs REST sobre HTTP [31].

A diferencia de las arquitecturas monolíticas tradicionales, donde todos los módulos forman parte de una única unidad de despliegue, los microservicios permiten desacoplar funcionalidades y distribuir las de forma independiente. Esta separación reduce el impacto de los cambios internos y facilita la evolución progresiva del sistema.

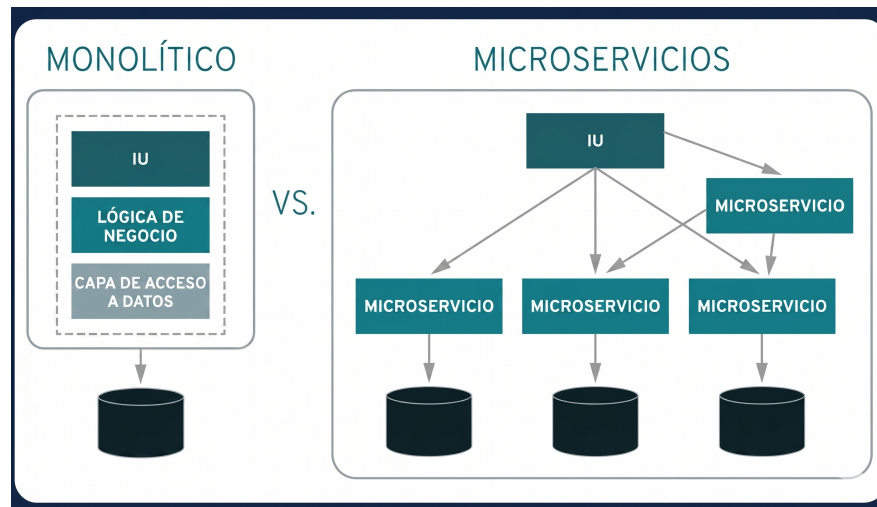


Figura 1.1: Comunicación entre microservicios.

Cada microservicio posee sus propios procesos de ejecución, mecanismos de persistencia y lógica de negocio, permitiendo implementar despliegues independientes y estrategias de escalabilidad específicas para cada componente. Esta característica resulta especialmente útil en sistemas distribuidos con cargas variables y requisitos de alta disponibilidad.

Entre las principales características de esta arquitectura destacan:

- **Autonomía funcional:** Cada servicio puede desarrollarse, desplegarse y mantenerse de forma independiente.
- **Desacoplamiento:** La separación de responsabilidades reduce la dependencia entre módulos y facilita la evolución tecnológica.
- **Escalabilidad independiente:** Los servicios pueden escalar horizontalmente según la demanda específica de procesamiento.
- **Resiliencia operativa:** Un fallo en un servicio no necesariamente compromete el funcionamiento global del sistema.
- **Interoperabilidad:** Permite integrar tecnologías heterogéneas mediante protocolos estándar de comunicación.

La principal ventaja de este modelo radica en la capacidad de construir sistemas altamente flexibles y escalables. Asimismo, facilita los ciclos de integración y despliegue continuo, permitiendo actualizar componentes individuales sin afectar el funcionamiento general de la aplicación [32].

No obstante, la arquitectura de microservicios también introduce desafíos relacionados con la complejidad operativa, la observabilidad distribuida, la gestión de redes, la sincronización de datos y el monitoreo de servicios. Debido a ello, su implementación suele requerir herramientas complementarias para el descubrimiento de servicios, balanceo de carga y tolerancia a fallos [32].

1.7.2 Arquitectura de tres capas

La arquitectura de tres capas es un modelo de organización de software que divide una aplicación en tres niveles lógicos independientes: presentación, lógica de negocio y persistencia de datos. Su propósito principal consiste en separar responsabilidades para reducir el acoplamiento entre componentes y facilitar el mantenimiento del sistema [33].

Este patrón arquitectónico es ampliamente utilizado en aplicaciones empresariales debido a que permite estructurar los procesos internos de manera organizada y modular. Cada capa interactúa únicamente con las capas adyacentes, favoreciendo la reutilización de componentes y la claridad estructural del sistema.

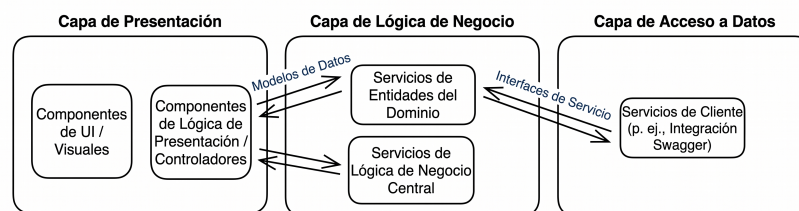


Figura 1.2: Patrón de arquitectura de tres capas.

Las capas que conforman esta arquitectura son las siguientes:

- **Capa de presentación:** Encargada de la interacción con el usuario o con sistemas externos. Su función consiste en recibir solicitudes, mostrar información y transmitir datos hacia la lógica de negocio.
- **Capa de lógica de negocio:** Constituye el núcleo funcional de la aplicación. En esta capa se implementan las reglas, validaciones y procesos que definen el comportamiento del sistema.
- **Capa de persistencia:** Responsable de la comunicación con los sistemas de almacenamiento de datos. Gestiona operaciones de acceso, recuperación y almacenamiento de información.

La separación estructural proporcionada por este modelo mejora significativamente la mantenibilidad y escalabilidad de las aplicaciones, ya que cada capa puede modificarse o evolucionar con un impacto mínimo sobre las demás [40].

Sin embargo, una segmentación excesiva también puede incrementar la complejidad del sistema y generar sobrecarga en la comunicación entre capas cuando no se diseña adecuadamente [40].

1.7.3 Servidor web (NGINX)

NGINX es un software de código abierto utilizado como servidor web, balanceador de carga y gestor de contenido estático. Su arquitectura basada en eventos y procesamiento asíncrono le permite manejar un gran número de conexiones concurrentes con un consumo reducido de memoria y recursos del sistema.

A diferencia de los servidores tradicionales basados en procesos o hilos por conexión, NGINX emplea un modelo no bloqueante orientado a eventos, lo que mejora considerablemente el rendimiento bajo escenarios de alta concurrencia.

Dentro de arquitecturas distribuidas y ecosistemas de microservicios, NGINX suele utilizarse como punto de entrada centralizado para gestionar el tráfico de red y redirigir solicitudes hacia los servicios internos correspondientes.

Entre sus principales funcionalidades destacan:

- **Balanceo de carga:** Distribuye solicitudes entre distintas instancias de servicios para mejorar la disponibilidad y el rendimiento.
- **Caché de contenido:** Reduce tiempos de respuesta mediante almacenamiento temporal de recursos frecuentes.
- **Gestión de tráfico concurrente:** Permite atender miles de conexiones simultáneas con alta eficiencia.

En arquitecturas modernas, NGINX también puede implementar el patrón *API Gateway*, actuando como intermediario entre clientes y servicios internos mediante mecanismos de autenticación, control de acceso y enrutamiento dinámico.

1.8 Tecnologías de Desarrollo

1.8.1 Lenguaje de programación: Java

Java es un lenguaje de programación orientado a objetos ampliamente utilizado en sistemas empresariales y aplicaciones distribuidas debido a su portabilidad, robustez y seguridad, permitiendo ejecutar aplicaciones sobre múltiples plataformas mediante la Java Virtual Machine (JVM).

La JVM actúa como una capa de abstracción entre el sistema operativo y el código compilado, proporcionando independencia de plataforma y mecanismos automáticos de administración de memoria. Esta arquitectura favorece la estabilidad y confiabilidad de aplicaciones de gran escala.

Entre las principales características técnicas de Java destacan:

- **Tipado fuerte:** Reduce errores durante la compilación y mejora la seguridad del código.
- **Gestión automática de memoria:** El recolector de basura administra dinámicamente la memoria utilizada por la aplicación.
- **Soporte multihilo:** Facilita el procesamiento concurrente y la ejecución paralela de tareas.
- **Portabilidad:** Las aplicaciones pueden ejecutarse en diferentes sistemas operativos sin modificaciones adicionales.
- **Amplio ecosistema empresarial:** Posee una gran cantidad de frameworks y herramientas para el desarrollo de sistemas distribuidos.

Debido a estas características, Java se ha consolidado como una de las tecnologías más utilizadas para aplicaciones backend de alta disponibilidad y arquitecturas empresariales distribuidas.

1.8.2 Framework: Spring Boot

Spring Boot es un framework perteneciente al ecosistema Spring orientado al desarrollo de aplicaciones empresariales y microservicios. Su principal objetivo consiste en simplificar la configuración y despliegue de aplicaciones Java mediante mecanismos de configuración automática y convenciones predefinidas [38].

Este framework adopta el principio *Convention over Configuration*, reduciendo significativamente la complejidad asociada a la configuración manual de componentes y dependencias.

Entre sus principales características destacan:

- **Configuración automática:** Detecta dependencias y configura componentes automáticamente según el entorno de ejecución.
- **Servidores embebidos:** Permite ejecutar aplicaciones sin necesidad de desplegar manualmente servidores externos.
- **Integración modular:** Facilita la incorporación de tecnologías de persistencia, seguridad y comunicación distribuida.
- **Desarrollo de APIs REST:** Proporciona herramientas para la construcción de servicios web y arquitecturas distribuidas.
- **Compatibilidad con Spring Ecosystem:** Permite integrar módulos especializados como Spring Security, Spring Data y Spring Cloud.

El núcleo conceptual del framework se fundamenta en los principios de Inversión de Control (IoC) e Inyección de Dependencias (DI), los cuales delegan la creación y administración de objetos al contenedor de Spring, reduciendo el acoplamiento entre componentes.

Spring Boot Actuator

Spring Boot Actuator es un módulo orientado a la observabilidad y monitoreo de aplicaciones en entornos de producción. Proporciona mecanismos para inspeccionar métricas, estados internos y comportamiento operativo de una aplicación sin necesidad de desarrollar herramientas adicionales [51].

Entre las funcionalidades más relevantes se encuentran:

- Supervisión del estado de salud de la aplicación.
- Exposición de métricas de rendimiento.
- Monitoreo del consumo de memoria y CPU.
- Gestión dinámica de registros de eventos (*logs*).
- Inspección de componentes internos administrados por Spring.

Estas capacidades permiten mejorar la observabilidad del sistema y facilitan la detección temprana de errores dentro de entornos distribuidos.

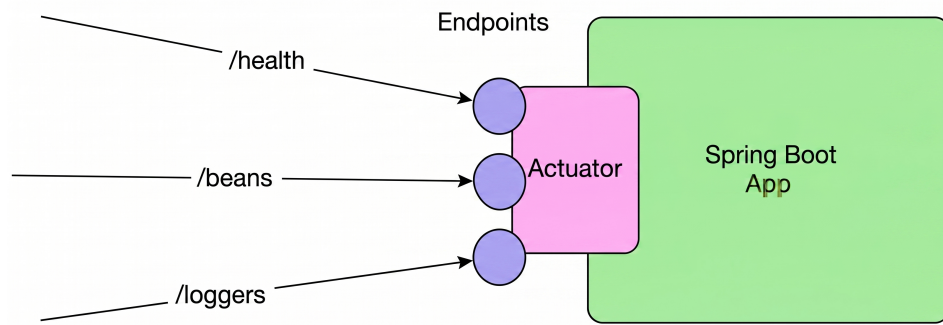


Figura 1.3: Endpoints de monitoreo proporcionados por Spring Boot Actuator.

Spring Security y JWT

Spring Security es un framework especializado en autenticación y autorización para aplicaciones Java. Su objetivo principal consiste en proteger recursos mediante mecanismos de control de acceso, validación de identidad y gestión de permisos.

Dentro de arquitecturas modernas distribuidas, es común utilizar estándares como JWT (*JSON Web Tokens*) para implementar autenticación basada en tokens [52].

JWT es un estándar de representación compacta de información firmado digitalmente, utilizado para transportar datos de autenticación y autorización entre sistemas.

El funcionamiento general del esquema basado en JWT consiste en:

1. El usuario se autentica mediante credenciales válidas.
2. El servidor genera un token firmado digitalmente.
3. El cliente almacena el token y lo envía en cada solicitud.
4. El servidor valida la autenticidad y vigencia del token antes de permitir el acceso.

Este enfoque permite construir sistemas *stateless*, reduciendo la dependencia de sesiones persistentes y facilitando la escalabilidad horizontal en arquitecturas distribuidas.

No obstante, el uso de JWT también introduce desafíos relacionados con la revocación de tokens, el manejo seguro de claves criptográficas y la protección contra ataques de reutilización de credenciales.

Spring Cloud: Configuración distribuida y descubrimiento de servicios

Spring Cloud es un conjunto de herramientas orientadas al desarrollo de sistemas distribuidos y arquitecturas basadas en microservicios. Su propósito consiste en proporcionar soluciones para problemas comunes en entornos distribuidos, tales como configuración centralizada, descubrimiento de servicios, balanceo de carga y tolerancia a fallos [53].

Configuración centralizada En sistemas distribuidos, mantener configuraciones independientes para cada servicio puede generar inconsistencias y dificultades de mantenimiento. Para resolver este problema, Spring Cloud Config permite centralizar propiedades y parámetros de configuración dentro de repositorios externos.

Este enfoque facilita la administración de múltiples entornos de ejecución, permitiendo separar configuraciones de desarrollo, pruebas y producción. Asimismo, favorece la actualización dinámica de parámetros sin necesidad de modificar directamente el código fuente de los servicios.

Descubrimiento de servicios Dentro de arquitecturas basadas en microservicios, las instancias pueden crearse, eliminarse o modificarse dinámicamente. Debido a ello, resulta necesario implementar mecanismos automáticos de localización de servicios.

Spring Cloud integra herramientas de *Service Discovery*, las cuales permiten registrar servicios dentro de un directorio centralizado. Los clientes pueden localizar servicios mediante nombres lógicos en lugar de depender de direcciones IP estáticas.

Entre las principales ventajas de este modelo destacan:

- Descubrimiento automático de servicios.
- Balanceo dinámico de carga.
- Reducción del acoplamiento entre componentes.
- Mayor tolerancia a fallos en sistemas distribuidos.
- Simplificación de la comunicación entre servicios.

No obstante, este tipo de arquitecturas también introduce complejidad adicional relacionada con la sincronización de estados, monitoreo distribuido y administración de infraestructura.

1.8.3 Interfaces de Programación de Aplicaciones (API REST)

Las Interfaces de Programación de Aplicaciones bajo el estilo arquitectónico REST (*Representational State Transfer*) constituyen uno de los modelos de comunicación más utilizados en sistemas distribuidos y arquitecturas basadas en servicios [54].

REST define un conjunto de principios para la transferencia de recursos mediante protocolos estándar de la web, particularmente HTTP. En este enfoque, cada recurso posee una dirección única identificada mediante una URI y puede manipularse utilizando operaciones HTTP como GET, POST, PUT y DELETE.

Entre las principales características de las APIs REST destacan:

- **Comunicación sin estado (*stateless*):** Cada solicitud contiene toda la información necesaria para ser procesada.
- **Interoperabilidad:** Facilita la comunicación entre sistemas heterogéneos mediante estándares abiertos.
- **Escalabilidad:** La ausencia de sesiones persistentes mejora el manejo de múltiples solicitudes concurrentes.
- **Separación cliente-servidor:** Permite desacoplar interfaces de usuario y servicios backend.
- **Uso de formatos estándar:** Generalmente emplea JSON o XML para el intercambio de información.

En arquitecturas modernas, las APIs REST suelen complementarse con estrategias de versionamiento para mantener compatibilidad entre distintas versiones de servicios. Asimismo, es común implementar mecanismos de control de tráfico (*Rate Limiting*) con el objetivo de evitar saturación y abuso de recursos.

1.8.4 Protocolos de Transporte Seguro: TLS 1.3

TLS (*Transport Layer Security*) es un protocolo criptográfico diseñado para garantizar confidencialidad, integridad y autenticación en comunicaciones realizadas a través de redes informáticas [55].

La versión TLS 1.3 representa una evolución significativa respecto a versiones anteriores, incorporando mejoras relacionadas con seguridad y rendimiento. Entre sus principales características destacan:

- **Reducción de latencia:** Optimiza el proceso de establecimiento de conexión mediante un menor número de intercambios durante el *handshake*.
- **Eliminación de algoritmos inseguros:** Suprime mecanismos criptográficos obsoletos o vulnerables presentes en versiones anteriores.
- **Perfect Forward Secrecy (PFS):** Garantiza que la exposición futura de claves privadas no comprometa comunicaciones previas.
- **Cifrado autenticado:** Utiliza algoritmos AEAD (*Authenticated Encryption with Associated Data*) para proteger confidencialidad e integridad simultáneamente.

Una de las mejoras más importantes de TLS 1.3 consiste en la reducción de la latencia durante el establecimiento de conexiones seguras mediante mecanismos como 1-RTT y 0-RTT [56].

No obstante, la implementación de protocolos criptográficos modernos también requiere una administración adecuada de certificados digitales, políticas de seguridad y actualización constante de dependencias criptográficas.

1.8.5 Resiliencia del Sistema: API Gateway

La resiliencia en sistemas distribuidos se refiere a la capacidad de una arquitectura para mantener su funcionamiento ante fallos parciales, interrupciones de red o sobrecarga de servicios [58].

Dentro de arquitecturas basadas en microservicios, es común implementar patrones especializados para mejorar la tolerancia a fallos y reducir el impacto de errores distribuidos.

API Gateway El patrón API Gateway actúa como un punto de entrada centralizado para las solicitudes realizadas por los clientes. Este componente abstrae la complejidad interna del sistema y centraliza funcionalidades como autenticación, balanceo de carga, monitoreo y enrutamiento dinámico.

Entre sus principales ventajas destacan:

- Simplificación de la comunicación cliente-servidor.
- Centralización de políticas de seguridad.
- Ocultamiento de la topología interna.
- Control unificado del tráfico de red.

- Integración de mecanismos de monitoreo y filtrado.

1.9 Infraestructura y Orquestación

La infraestructura computacional moderna se fundamenta en mecanismos de abstracción y virtualización que permiten optimizar el uso de recursos físicos, mejorar la portabilidad de las aplicaciones y facilitar la administración de entornos distribuidos. En arquitecturas basadas en microservicios, la infraestructura debe ser capaz de soportar despliegues dinámicos, escalabilidad horizontal y recuperación automática ante fallos.

Dentro de este contexto, tecnologías como la virtualización, la contenerización y la orquestación de contenedores constituyen elementos fundamentales para la construcción de sistemas distribuidos de alta disponibilidad.

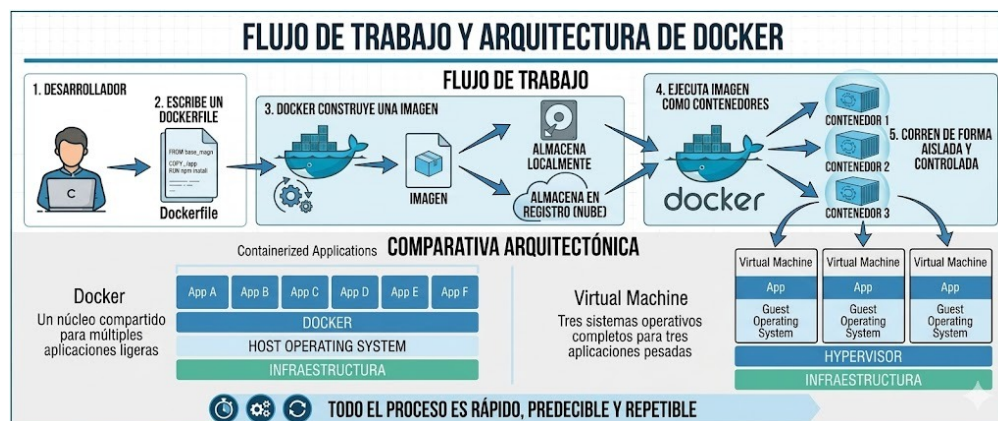


Figura 1.4: Funcionamiento de Docker y Kubernetes en la infraestructura de microservicios.

1.9.1 Virtualización e Hipervisores

La virtualización es una técnica computacional que permite ejecutar múltiples entornos aislados sobre una misma infraestructura física mediante la abstracción del hardware subyacente. Este enfoque incrementa el aprovechamiento de recursos y facilita la administración de servidores y aplicaciones.

El componente principal encargado de gestionar este proceso es el **hipervisor** (*Hypervisor*), una capa de software especializada que administra el acceso al hardware físico y permite crear y ejecutar múltiples máquinas virtuales de manera simultánea.

Existen dos categorías principales de hipervisores:

- **Hipervisores Tipo 1 (Bare Metal):** Se ejecutan directamente sobre el hardware físico sin requerir un sistema operativo anfitrión. Ejemplos comunes incluyen VMware ESXi, Xen y Microsoft Hyper-V.
- **Hipervisores Tipo 2 (Hosted):** Operan sobre un sistema operativo anfitrión, utilizando sus recursos para ejecutar máquinas virtuales. Entre los más utilizados se encuentran VirtualBox y VMware Workstation.

La virtualización proporciona ventajas importantes relacionadas con aislamiento, consolidación de servidores y flexibilidad operativa. Sin embargo, las máquinas virtuales suelen requerir una mayor cantidad de recursos debido a que cada instancia incorpora un sistema operativo completo.

1.9.2 Máquinas Virtuales y Sistema Operativo Anfitrión

Una **máquina virtual** (*Virtual Machine*) es una representación virtualizada de un sistema computacional completo, incluyendo procesador, memoria, almacenamiento y sistema operativo. Cada máquina virtual opera de manera aislada respecto a las demás, como si se tratara de un equipo físico independiente.

Dentro de este modelo intervienen dos conceptos fundamentales:

- **Host Operating System:** Sistema operativo anfitrión encargado de administrar directamente el hardware físico del servidor o estación de trabajo.
- **Guest Operating System:** Sistema operativo invitado que se ejecuta dentro de una máquina virtual mediante la capa de virtualización proporcionada por el hipervisor.

Este enfoque permite ejecutar múltiples sistemas operativos simultáneamente sobre un mismo hardware físico, facilitando tareas de desarrollo, pruebas, consolidación de infraestructura y despliegue de servicios distribuidos.

No obstante, debido a que cada máquina virtual incorpora su propio núcleo del sistema operativo y recursos dedicados, el consumo de memoria y procesamiento puede incrementarse significativamente en comparación con otras alternativas más ligeras.

1.9.3 Contenerización

La contenerización es un modelo de virtualización ligera que permite empaquetar aplicaciones junto con sus dependencias, librerías y configuraciones necesarias para su ejecución dentro de unidades aisladas denominadas contenedores.

A diferencia de las máquinas virtuales tradicionales, los contenedores no requieren un sistema operativo completo para cada instancia. En su lugar, comparten el núcleo del sistema operativo anfitrión, reduciendo significativamente el consumo de recursos y mejorando los tiempos de despliegue.

Este modelo proporciona ventajas importantes relacionadas con:

- Portabilidad entre entornos.
- Aislamiento de aplicaciones.
- Reproducibilidad de configuraciones.
- Escalabilidad rápida.
- Reducción del consumo de infraestructura.

La contenerización se ha convertido en uno de los pilares fundamentales de las arquitecturas basadas en microservicios debido a su capacidad para simplificar despliegues distribuidos y mantener consistencia entre entornos de desarrollo, pruebas y producción.

1.9.4 Docker

Docker es una plataforma de código abierto orientada a la creación, distribución y ejecución de contenedores [58]. Su arquitectura permite encapsular aplicaciones junto con todas sus dependencias dentro de entornos aislados y portables.

El funcionamiento de Docker se basa en tecnologías del núcleo de Linux como *Namespaces* y *Control Groups (cgroups)*, las cuales permiten aislar procesos y controlar el consumo de recursos de cada contenedor.

Entre los principales componentes del ecosistema Docker destacan:

- **Docker Engine:** Motor principal encargado de construir y ejecutar contenedores.

- **Dockerfile:** Archivo declarativo que contiene las instrucciones necesarias para construir imágenes de contenedores.
- **Imágenes:** Plantillas inmutables que incluyen dependencias, librerías y configuraciones requeridas para ejecutar aplicaciones.
- **Contenedores:** Instancias ejecutables creadas a partir de imágenes Docker.
- **Docker Registry:** Repositorio utilizado para almacenar y distribuir imágenes.
- **Docker Compose:** Herramienta destinada a definir y administrar aplicaciones compuestas por múltiples contenedores.

Una de las principales ventajas de Docker consiste en garantizar consistencia operativa entre diferentes entornos de ejecución. Esto permite que una aplicación pueda ejecutarse de manera idéntica independientemente de la infraestructura subyacente.

Sin embargo, el uso masivo de contenedores también introduce desafíos relacionados con monitoreo, seguridad, redes distribuidas y administración de múltiples servicios concurrentes.

1.9.5 Orquestación de Contenedores con Kubernetes

Kubernetes es una plataforma de código abierto diseñada para automatizar el despliegue, administración y escalabilidad de aplicaciones contenerizadas [59].

Su principal objetivo consiste en coordinar múltiples contenedores distribuidos dentro de un clúster computacional, garantizando disponibilidad, balanceo de carga y recuperación automática ante fallos.

Kubernetes implementa un modelo declarativo de infraestructura, donde el usuario define el estado deseado del sistema y la plataforma se encarga automáticamente de mantener dicha configuración.

Entre las principales abstracciones utilizadas por Kubernetes destacan:

- **Pods:** Unidad mínima de ejecución que agrupa uno o más contenedores relacionados.
- **Deployments:** Recursos encargados de administrar el estado deseado de las aplicaciones y sus actualizaciones.
- **Services:** Componentes que proporcionan conectividad y balanceo de carga entre Pods.
- **Namespaces:** Mecanismos de aislamiento lógico dentro del clúster.

- **Horizontal Pod Autoscaler (HPA):** Sistema de escalabilidad automática basado en métricas de utilización.

Además de la orquestación de contenedores, Kubernetes incorpora funcionalidades relacionadas con:

- Recuperación automática ante fallos.
- Distribución dinámica de tráfico.
- Escalabilidad horizontal.
- Gestión declarativa de infraestructura.
- Monitoreo y supervisión de servicios.
- Balanceo de carga distribuido.

Estas capacidades convierten a Kubernetes en una de las principales plataformas para la administración de infraestructuras modernas basadas en microservicios y computación distribuida.

No obstante, la administración de clústeres distribuidos también introduce complejidad significativa relacionada con redes, observabilidad, seguridad, almacenamiento persistente y consumo de recursos computacionales.

1.10 Gestión de Bases de Datos

La gestión de bases de datos constituye uno de los componentes fundamentales dentro de los sistemas distribuidos modernos, ya que permite almacenar, organizar, consultar y proteger grandes volúmenes de información de manera eficiente.

En arquitecturas basadas en microservicios, los sistemas gestores de bases de datos deben garantizar propiedades relacionadas con integridad, concurrencia, disponibilidad, seguridad y escalabilidad, permitiendo que múltiples servicios interactúen simultáneamente sobre los datos sin comprometer su consistencia.

1.10.1 PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacional orientado a objetos (*Object Relational Database Management System, ORDBMS*) de código abierto, ampliamente reconocido por su robustez, estabilidad y cumplimiento de estándares SQL [42].

Su arquitectura permite combinar características tradicionales de los sistemas relacionales con capacidades avanzadas orientadas al manejo de datos complejos y aplicaciones distribuidas.

A diferencia de motores ligeros enfocados únicamente en almacenamiento relacional básico, PostgreSQL incorpora mecanismos avanzados de concurrencia, indexación, recuperación ante fallos y extensibilidad, lo que lo convierte en una solución ampliamente utilizada en aplicaciones empresariales, plataformas web y sistemas de misión crítica.

Entre sus principales características destacan:

- **Cumplimiento de propiedades ACID:** Garantiza atomicidad, consistencia, aislamiento y durabilidad en las transacciones, asegurando que las operaciones sobre los datos se ejecuten de forma íntegra y confiable.
- **Control de concurrencia multiversión (MVCC):** Permite que múltiples transacciones accedan simultáneamente a la información sin bloquear completamente las tablas, mejorando el rendimiento en entornos concurrentes.
- **Soporte para transacciones complejas:** Incluye subconsultas, procedimientos almacenados, disparadores (*triggers*) y operaciones avanzadas de manipulación de datos.
- **Extensibilidad:** Permite incorporar tipos de datos personalizados, funciones definidas por el usuario y extensiones adicionales.
- **Compatibilidad con múltiples índices avanzados:** Soporta estructuras de indexación como B-Tree, Hash, GiST y GIN.
- **Soporte híbrido de datos:** Permite trabajar tanto con modelos relacionales tradicionales como con datos semi-estructurados mediante JSON y JSONB.

Gracias a estas capacidades, PostgreSQL se ha consolidado como uno de los sistemas gestores de bases de datos más utilizados en arquitecturas modernas que requieren alta integridad, confiabilidad y capacidad de procesamiento concurrente.

1.10.2 Manejo de datos semi-estructurados mediante JSONB

Los sistemas actuales generan grandes cantidades de información cuya estructura puede variar dinámicamente dependiendo del contexto de la aplicación. Para responder a esta necesidad, PostgreSQL incorpora soporte para datos semi-estructurados mediante los tipos JSON y JSONB.

El tipo **JSONB** (*JSON Binary*) almacena documentos JSON en un formato binario optimizado, permitiendo realizar búsquedas, filtrados e indexación de manera más eficiente que el tipo JSON convencional [60].

A diferencia del almacenamiento textual tradicional, JSONB descompone internamente la estructura del documento, facilitando operaciones de consulta y acceso rápido a los atributos almacenados.

Entre las principales ventajas de JSONB destacan:

- **Flexibilidad en el modelado de datos:** Permite almacenar estructuras dinámicas sin necesidad de modificar constantemente el esquema relacional.
- **Consultas eficientes:** Facilita búsquedas sobre atributos específicos dentro de documentos complejos.
- **Compatibilidad con operadores especializados:** PostgreSQL incorpora operadores para validar existencia de claves, comparar estructuras y filtrar contenido interno.
- **Integración con mecanismos avanzados de indexación:** Mejora significativamente el rendimiento en consultas sobre grandes volúmenes de documentos.

El uso de datos semi-estructurados resulta especialmente útil en sistemas que manejan metadatos variables, registros dinámicos, bitácoras, configuraciones o documentos cuyo contenido no sigue una estructura completamente rígida.

1.10.3 Índices GIN y búsqueda eficiente de información

En bases de datos con grandes volúmenes de información, las operaciones de búsqueda pueden convertirse en un cuello de botella si no existen mecanismos adecuados de indexación.

Para optimizar este tipo de consultas, PostgreSQL incorpora estructuras avanzadas conocidas como índices **GIN** (*Generalized Inverted Index*) [60].

Un índice GIN funciona mediante una estructura invertida que asocia cada término, clave o elemento almacenado con las filas donde aparece dicho contenido. Este enfoque resulta especialmente eficiente para:

- Consultas sobre documentos JSONB.
- Búsquedas de texto completo (*Full-Text Search*).
- Filtrado sobre arreglos y colecciones de datos.
- Consultas sobre estructuras dinámicas o semi-estructuradas.

A diferencia de índices tradicionales como B-Tree, los índices GIN están diseñados específicamente para manejar atributos múltiples dentro de una misma estructura de datos, permitiendo acelerar considerablemente las búsquedas complejas.

Gracias a ello, PostgreSQL puede realizar consultas sobre grandes volúmenes de información textual y documental manteniendo tiempos de respuesta eficientes.

1.10.4 Seguridad y cifrado de datos en reposo

La protección de información almacenada constituye uno de los principales requisitos dentro de sistemas empresariales y aplicaciones distribuidas, especialmente cuando se gestionan datos sensibles o confidenciales.

La seguridad de datos en reposo se refiere al conjunto de mecanismos destinados a proteger la información almacenada físicamente dentro de discos, particiones o sistemas de almacenamiento persistente.

Para ello, PostgreSQL proporciona diversas estrategias relacionadas con autenticación, control de acceso y cifrado de información mediante extensiones especializadas como `pgcrypto` [60].

Entre las principales técnicas de protección destacan:

- **Cifrado simétrico AES-256:** Utiliza claves criptográficas para proteger información sensible mediante algoritmos de cifrado robustos y ampliamente adoptados en entornos empresariales.
- **Funciones hash SHA-512:** Permiten generar resúmenes criptográficos únicos utilizados para verificar integridad y detectar modificaciones no autorizadas sobre los datos.

- **Control de acceso basado en roles:** PostgreSQL implementa un modelo de permisos jerárquico que restringe operaciones según privilegios asignados a usuarios y grupos.
- **Autenticación y políticas de acceso:** El sistema permite definir mecanismos de autenticación mediante contraseñas cifradas, certificados y reglas de acceso configurables.

Estas estrategias fortalecen la confidencialidad, integridad y disponibilidad de la información almacenada, reduciendo riesgos relacionados con accesos no autorizados y alteraciones maliciosas.

1.10.5 Escalabilidad y optimización de bases de datos

A medida que aumenta el volumen de información y el número de usuarios concurrentes, resulta necesario implementar mecanismos que permitan mantener el rendimiento y la estabilidad del sistema gestor de bases de datos.

La escalabilidad en bases de datos se refiere a la capacidad de incrementar recursos y distribuir carga de trabajo sin degradar significativamente el desempeño de las operaciones.

Entre las principales estrategias utilizadas para optimización y escalabilidad destacan:

- **Particionamiento de tablas (*Partitioning*):** Técnica que divide grandes volúmenes de datos en segmentos más pequeños organizados por rangos, listas o criterios específicos, reduciendo tiempos de búsqueda y acceso.
- **Pooling de conexiones:** Herramientas como **PgBouncer** permiten reutilizar conexiones activas hacia PostgreSQL, disminuyendo el costo computacional asociado a la apertura constante de nuevas conexiones.
- **Replicación Streaming:** Permite mantener múltiples réplicas sincronizadas de una base de datos principal, mejorando disponibilidad, tolerancia a fallos y distribución de carga.
- **Balanceo de consultas:** Las operaciones de lectura pueden distribuirse entre servidores secundarios para reducir carga sobre el nodo principal.
- **Optimización mediante índices:** El uso adecuado de estructuras de indexación reduce significativamente el tiempo requerido para ejecutar consultas complejas.

Estas estrategias permiten incrementar la capacidad de procesamiento, mejorar la disponibilidad y mantener un comportamiento estable incluso bajo escenarios de alta demanda concurrente.

Bibliografía

- [1] Instituto Politécnico Nacional, “*Ley Orgánica del Instituto Politécnico Nacional*”, Diario Oficial de la Federación, Ciudad de México, México. Disponible en: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/ley-org%C3%A1nica-ipn.pdf>
- [2] Instituto Politécnico Nacional, “*Reglamento Interno del Instituto Politécnico Nacional*”, Gaceta Politécnica, Ciudad de México, México. Disponible en: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/reglamento-interno.pdf>
- [3] Instituto Politécnico Nacional, “*Acuerdo de creación de la Defensoría de los Derechos Politécnicos*”, Gaceta Politécnica, no. 622 (Extraordinaria), pp. 3-7, 31 ene. 2006. Disponible en: <https://www.ipn.mx/assets/files/imageninstitucional/docs/gaceta-extraordinaria/2006/01/622-31-enero-gaceta-extra-.pdf>
- [4] Instituto Politécnico Nacional, “*Manual de Organización de la Defensoría de los Derechos Politécnicos*”, Dirección de Planeación y Organización, Ciudad de México, México, oct. 2022. Disponible en: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/modd2022.pdf>
- [5] Instituto Politécnico Nacional, “*Manual de Procedimientos de la Defensoría de los Derechos Politécnicos*”, Dirección de Planeación y Organización, Ciudad de México, México, oct. 2024. Disponible en: <https://www.ipn.mx/assets/files/defensoria/docs/manuales/mp-ddp.pdf>
- [6] Secretaría de la Función Pública, “*Sistema Integral de Denuncias Ciudadanas (SIDECE)*”, Gobierno de México. Disponible en: <https://sidec.buengobierno.gob.mx/>
- [7] Agencia Mexicana de Cooperación Internacional para el Desarrollo (AMEXCID), “*Catálogo de Capacidades Mexicanas: Sistema Integral de Quejas y Denuncias Ciudadanas (SIDECE)*”, SRE. Disponible en: <https://de.sre.gob.mx/capacidades/instituciones-mexicanas/sfp/sistema-integral-de-quejas-y-denuncias-ciudadanas-sidec>

-
- [8] Secretaría Anticorrupción y Buen Gobierno, “*Implementan SFP, BID y GovLab proyecto de mejora al Sistema Integral de Denuncias Ciudadanas (SIDECA)*”, Gob.mx, 22 jul. 2018. Disponible en: <https://www.gob.mx/buengobierno/prensa/implementan-sfp-bid-y-govlab-proyecto-de-mejora-al-sistema-integral-de-denuncias-ciudadanas-sidec>
- [9] Gobierno de la Ciudad de México, “*Denuncia Digital*”, Fiscalía General de Justicia de la CDMX. Disponible en: <https://denunciadigital.cdmx.gob.mx/>
- [10] Denuncia.org, “*Denuncia en línea: Consulta si en tu estado la Fiscalía cuenta con la opción de denuncia digital*”, Impunidad Cero, 2026. Disponible en: <https://denuncia.org/denuncia-digital/>
- [11] Fiscalía General de Justicia de la Ciudad de México, “*Servicios en Línea: Denuncia Digital, Denuncia Anónima y MP Transparente*”, FGJ-CDMX, 2026. Disponible en: <https://www.fgjcdmx.gob.mx/servicios/en-linea>
- [12] Gobierno de la Ciudad de México, “*MP Virtual: Ratificación de Querellas y Actas Especiales*”, FGJ-CDMX. Disponible en: <https://mpvirtual.fgjcdmx.gob.mx/CiberDenuncia/TerminosCondiciones.aspx>
- [13] Nación321, “*5 apps que nos ayudan a combatir la corrupción*”, Grupo Multimedia Lauman, SAPI de CV, 16 feb. 2024. Disponible en: <https://www.nacion321.com/ciudadanos/5-apps-que-nos-ayudan-a-combatir-la-corrupcion>
- [14] Secretaría de la Contraloría del Gobierno del Estado de México, “*Sistema de Atención Mexiquense (SAM)*”, GEM, 2026. Disponible en: <https://www.secogem.gob.mx/sam/tramite.asp>
- [15] Whistlelink, “*The All-in-One Whistleblowing Solution*”, Whistlelink.com, 2026. Disponible en: <https://www.whistlelink.com/>
- [16] EQS Group, “*EQS Integrity Line: The leading whistleblowing system in Europe*”, EQS.com, 2026. Disponible en: <https://www.eqs.com/es/soluciones-gobierno-corporativo/integrity-line/>
- [17] NAVEX Global, “*WhistleB: Whistleblowing Solutions for a Modern Workplace*”, Navex.com, 2026. Disponible en: <https://www.navex.com/en-us/products/whistleb/>

- [18] Whistleblower Software by Whistleblowing Solutions ApS, “*The simple, secure, and compliant whistleblowing system*”, Whistleblowersoftware.com, 2026. Disponible en: <https://whistleblowersoftware.com/es>
- [19] Lefebvre, “*Centinela: Canal de denuncias para el cumplimiento normativo*”, Lefebvre.es, 2026. Disponible en: <https://lefebvre.es/soluciones/centinela/canal-denuncias/>
- [20] Coloriuris, “*Canal de Denuncias: Cumplimiento de la Ley 2/2023 y Directiva (UE) 2019/1937*”, Coloriuris.net, 2026. Disponible en: <https://www.coloriuris.net/canal-de-denuncias/>
- [21] Bully Button, “*The Bully Button: Reporting and Prevention System*”, BullyButton.com, 2026. Disponible en: <http://bullybutton.com/>
- [22] Secretaría de Educación Pública, “*Ventanilla Escolar: Orientación y Quejas*”, Gobierno de México. Disponible en: <https://www.gob.mx/sep/acciones-y-programas/ventanilla-escolar>
- [23] Instituto Politécnico Nacional, “*Protocolo Alerta IPN: Seguridad Institucional*”, Secretaría General del IPN. Disponible en: <https://www.ipn.mx/seguridad/>
- [24] Universidad Nacional Autónoma de México, “*Sistema de Denuncia en Línea de la Defensoría de los Derechos Universitarios*”, UNAM. Disponible en: <https://www.defensoria.unam.mx/denuncia/>
- [25] Procuraduría Federal de la Defensa del Trabajo, “*PROFEDET Digital: Servicios de orientación y seguimiento*”, Gobierno de México. Disponible en: <https://www.gob.mx/profedet>
- [26] Instituto Politécnico Nacional, “*Protocolo para la Prevención, Detección, Atención y Sanción de la Violencia de Género en el Instituto Politécnico Nacional*”, Gaceta Politécnica, Número 1726, Ciudad de México, México, mayo 2023.
- [27] BBVA, “*Arquitectura de software o sistemas: ¿qué es y ejemplos?*”, bbva.com, 2021. [En línea]. Disponible: <https://www.bbva.com/es/innovacion/arquitectura-de-software-o-sistemas-que-es-y-ejemplos/>. [Accedido: 10-abr-2026].
- [28] UTEL, “*Atributos de calidad del software*”, SCALA. [En línea]. Disponible: https://gc.scalahed.com/recursos/files/r161r/w26023w/L1IDS118_l2_s3.pdf. [Accedido: 10-abr-2026].

- [29] Microsoft, “¿Qué es la escalabilidad?”, Microsoft Learn, 2023. [En línea]. Disponible: <https://learn.microsoft.com/es-es/biztalk/core/what-is-scalability>. [Accedido: 10-abr-2026].
- [30] IBM, “¿Qué es una arquitectura monolítica?”, ibm.com. [En línea]. Disponible: <https://www.ibm.com/think/topics/monolithic-architecture>. [Accedido: 10-abr-2026].
- [31] Amazon Web Services (AWS), “Construcción de una arquitectura distribuida y escalable”, aws.amazon.com. [En línea]. Disponible: <https://aws.amazon.com/startups/learn/building-scalable-distributed-architecture?lang=es>. [Accedido: 10-abr-2026].
- [32] Amazon Web Services (AWS), “¿Qué son los microservicios?”, aws.amazon.com. [En línea]. Disponible: <https://aws.amazon.com/es/microservices/>. [Accedido: 10-abr-2026].
- [33] IBM, “¿Qué es la arquitectura de tres capas?”, ibm.com. [En línea]. Disponible: <https://www.ibm.com/mx-es/think/topics/three-tier-architecture>. [Accedido: 10-abr-2026].
- [34] NGINX, “nginx”, nginx.org. [En línea]. Disponible: <https://nginx.org/en/>. [Accedido: 10-abr-2026].
- [35] B. Khanal, “Nginx: The Lightweight Powerhouse for Modern Web Serving”, Medium, 2023. [En línea]. Disponible: <https://medium.com/@bishal.khanal.2057/nginx-the-lightweight-powerhouse-for-modern-web-serving-717f7cc628da>. [Accedido: 10-abr-2026].
- [36] Oracle, “¿Qué es la tecnología Java y para qué la necesito?”, java.com. [En línea]. Disponible: https://www.java.com/es/download/help/whatis_java.html. [Accedido: 10-abr-2026].
- [37] IBM, “¿Qué es Java?”, ibm.com. [En línea]. Disponible: <https://www.ibm.com/mx-es/think/topics/java>. [Accedido: 10-abr-2026].
- [38] IBM, “¿Qué es Spring Boot?”, ibm.com. [En línea]. Disponible: <https://www.ibm.com/mx-es/think/topics/java-spring-boot>. [Accedido: 10-abr-2026].
- [39] Amazon Web Services (AWS), “¿Qué es Python?”, aws.amazon.com. [En línea]. Disponible: <https://aws.amazon.com/es/what-is/python/>. [Accedido: 10-abr-2026].
- [40] Angular, “What is Angular?”, angular.dev. [En línea]. Disponible: <https://angular.dev/overview>. [Accedido: 10-abr-2026].

- [41] Imagina Formación, “*Conoce las ventajas de utilizar Angular*”, imaginaformacion.com. [En línea]. Disponible: <https://imaginaformacion.com/tutoriales/conoce-las-ventajas-de-utilizar-angular>. [Accedido: 10-abr-2026].
- [42] IBM, “*¿Qué es PostgreSQL?*”, ibm.com. [En línea]. Disponible: <https://www.ibm.com/mx-es/think/topics/postgresql>. [Accedido: 10-abr-2026].
- [43] Red Hat, “*¿Qué es Docker?*”, redhat.com. [En línea]. Disponible: <https://www.redhat.com/es/topics/containers/what-is-docker>. [Accedido: 10-abr-2026].
- [44] GitHub, “*Acerca de GitHub y Git*”, docs.github.com. [En línea]. Disponible: <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>. [Accedido: 10-abr-2026].
- [45] E. Linis, “*Conceptos Básicos de Git*”, Gist GitHub. [En línea]. Disponible: <https://gist.github.com/erlinis/57a55dfb0337f5cd15cd>. [Accedido: 10-abr-2026].
- [46] Villamizar, M., et al. *Evaluating the Monolithic and the Microservice Architecture Pattern to Deploy Web Applications in the Cloud*. Universidad de los Andes, Colombia.
- [47] Tanenbaum, A. S., et al. *Arquitectura de Sistemas Distribuidos*.
- [48] Microsoft Ibérica. *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0*.
- [49] Poza Luján, J. L. *Propuesta de arquitectura distribuida de control inteligente basada en políticas de calidad de servicio*. Universitat Politècnica de València.
- [50] Coti Colop, B. M. *Reglas del Negocio en Arquitectura Tres Capas*. Universidad de San Carlos de Guatemala.
- [51] Álvarez Caules, C. *Spring Boot Actuator*. Arquitectura Java. Recuperado de: <https://www.arquitecturajava.com/spring-boot-actuator/>
- [52] Spring Security. *OAuth 2.0 Resource Server JWT*. VMware, Inc. Recuperado de: <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>
- [53] Spring Cloud. *Spring Cloud Config & Netflix Eureka*. VMware, Inc. Recuperado de: <https://spring.io/projects/spring-cloud>
- [54] Fielding, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.

- [55] Kleppmann, M. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- [56] Rescorla, E. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, Internet Engineering Task Force (IETF).
- [57] Stopford, B. *Designing Event-Driven Systems: Concepts and Patterns for Streaming Services with Apache Kafka*. O'Reilly Media.
- [58] Nygard, M. T. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf.
- [59] Burns, B., Beda, J., y Hightower, K. *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. O'Reilly Media.
- [60] Schönig, H. J. *Mastering PostgreSQL 15: Advanced techniques to build and manage scalable, reliable, and fault-tolerant database applications*. Packt Publishing.
- [61] C. Cortes y V. Vapnik, “Support-Vector Networks”, *Machine Learning*, vol. 20, no. 3, pp. 273–297, sep. 1995. DOI: <https://doi.org/10.1007/BF00994018>
- [62] L. Breiman, “Random Forests”, *Machine Learning*, vol. 45, no. 1, pp. 5–32, oct. 2001. DOI: <https://doi.org/10.1023/A:1010933404324>
- [63] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin, “Attention is All You Need”, en *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pp. 5998–6008, 2017. Disponible en: <https://arxiv.org/abs/1706.03762>
- [64] N. Reimers e I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”, en *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, Hong Kong, China, nov. 2019. Disponible en: <https://arxiv.org/abs/1908.10084>
- [65] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado y J. Dean, “Distributed Representations of Words and Phrases and their Compositionality”, en *Advances in Neural Information Processing Systems 26 (NIPS 2013)*, pp. 3111–3119, 2013. Disponible en: <https://arxiv.org/abs/1310.4546>
- [66] M. Honnibal, I. Montani, S. Van Landeghem y A. Boyd, “spaCy: Industrial-strength Natural Language Processing in Python”, Zenodo, 2020. DOI: <https://doi.org/10.5281/zenodo.1212303>

- [67] F. Pedregosa *et al.*, “*Scikit-learn: Machine Learning in Python*”, *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Disponible en: <http://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>